



NAME : HAMDAN ALI

WEEK 3 : Day 14

DAY 14

Create a shopping list array and display all items using a loop.

Task 14 : Screenshoots Of Output

WEEK 3 - DAY 14 · ARRAYS & LOOPS

The Grocery Ledger

Every item, quantity, and price below is live — edit it, add a line, cross one out. The array updates as you go.

This Week's List

Reset to defaults

Click any field to edit it. Changes are saved automatically, so the list is still here next time you open this page.

#	ITEM	QTY	PRICE	LINE TOTAL	
1	Basmati Rice	2 kg	Rs. 380	Rs. 760	✕
2	Cooking Oil	1 liter	Rs. 650	Rs. 650	✕
3	Red Lentils (Masc	1 kg	Rs. 320	Rs. 320	✕
4	Chicken	1.5 kg	Rs. 950	Rs. 1,425	✕
5	Tomatoes	2 kg	Rs. 240	Rs. 480	✕
6	Onions	3 kg	Rs. 270	Rs. 810	✕
7	Milk	2 liter	Rs. 340	Rs. 680	✕
8	Tea Leaves	1 pack	Rs. 410	Rs. 410	✕
9	Item name	1 unit	Rs. 0	Rs. 0	✕

+ ADD A LINE ITEM

RECEIPT SUMMARY

EST. 2026

Items on list9

Average per itemRs. 615

TOTAL DUE

Rs. 5,535

How it's built

A `for` loop draws every row from the `shoppingList` array. Editing a field updates that same array directly and a `while` loop recalculates the total — no page reload needed.

Add Item Aslo Manually :

9	Mango	4	unit	Rs.	400	Rs. 1,600	×
+ ADD A LINE ITEM							

EXPLANATION

DAY 14 — ARRAYS AND LOOPS (REVISED)

The Grocery Ledger — An Editable Shopping List

Hamdan Ali · Week 3 · Full Stack Web Development Internship

Why I Went Back and Reworked This One

The first version of Day 14 satisfied the task — an array, a for loop, a while loop, everything displayed on the page. But it was read-only. Nothing on it could be changed without opening script.js and editing the array by hand, which isn't how a real shopping list works. So I rebuilt it as something a person could actually use: every item, quantity, and price is now a live field, items can be added or removed, and the totals recalculate immediately. I also gave it a proper visual identity instead of reusing the same card-and-navbar layout from the earlier days.

The Design Direction — "The Grocery Ledger"

Rather than a generic dashboard look, I designed this one around what a shopping list actually is in real life — closer to a shopkeeper's ledger book or a till receipt than a typical web app screen. The page opens with a dark green "cover," the list itself sits on cream ledger paper with faint ruled lines and a red margin rule down the side (the kind you'd see on real accounting paper), and every price is set in a monospace type that reads like numbers punched into an old cash register.

The one deliberately bold piece is the summary panel on the right: it's shaped like a torn receipt, with a jagged bottom edge cut using CSS clip-path, and it updates live as the ledger changes — total cost, item count, and average price per item, all in that same register-style monospace. Everything else on the page stays quiet and out of its way, which is the idea behind spending the "bold" choice in one place rather than spreading effects everywhere.

Typography: Fraunces for headings (a serif with enough character to feel like a ledger book's cover lettering), Work Sans for the everyday reading text, and Space Mono for every number — quantities, prices, totals — so figures always look distinct from the item names sitting next to them.

Making Every Field Editable

Each row in the array is now rendered as a table row with real `<input>` fields instead of plain text:

```
<input class="field field-item" value="{entry.item}">
<input class="field field-qty" value="{entry.quantity}">
<input class="field field-price" value="{entry.price}">
```

Typing in any of these fires an input event that updates that exact object inside the shoppingList array — not a copy, the real thing the rest of the app reads from:

```
row.querySelector('.field-price').addEventListener('input', (e) => {
  updateEntry(entry.id, { price: parseFloat(e.target.value) || 0 });
  refreshRowTotal(row, entry.id);
});
```

The tricky part here was making sure typing didn't feel broken. My first instinct was to just re-render the whole table on every keystroke, the same `renderLedger()` function used for the initial load — but that rebuilds every input from scratch, which knocks the cursor out of the field the moment you type a second character. So editing a field now only updates the array and repaints

that one row's total and the summary panel, while add and delete — actual structural changes — are the only actions that trigger a full re-render.

Adding and Removing Items

Adding a row pushes a blank entry onto the array, re-renders, and immediately focuses the new row's name field so typing can start right away:

```
function addEntry() {  
  const newEntry = { id: nextId++, item: "", quantity: 1, unit: 'unit', price: 0 };  
  shoppingList.push(newEntry);  
  renderLedger();  
  newRow.querySelector('.field-item').focus();  
}
```

Removing one filters it out by id and redraws:

```
function removeEntry(id) {  
  shoppingList = shoppingList.filter((e) => e.id !== id);  
  renderLedger();  
}
```

Each item carries its own id (a plain incrementing number) precisely so it can be found and removed correctly even after the array has been reordered or edited — matching by array position alone gets unreliable the moment items are added or deleted.

The for Loop and while Loop Are Still Doing the Original Work

The core loop logic from the first version didn't change, it just now runs every time the array changes instead of once. A for loop still builds the table, row by row:

```
for (let i = 0; i < shoppingList.length; i++) {  
  const entry = shoppingList[i];  
  ledgerBody.appendChild(buildRow(entry, i + 1));  
}
```

And a while loop still adds up the total — now multiplying quantity by price per line, since totals need to reflect whatever the user has typed in at that moment:

```
let total = 0;
let index = 0;

while (index < shoppingList.length) {
  total += shoppingList[index].quantity * shoppingList[index].price;
  index++;
}
```

Remembering the List Between Visits

One more addition: the list now saves itself to the browser's localStorage every time it changes, and loads that saved version back in when the page opens again. This wasn't part of the original task, but an editable list that forgets everything on refresh isn't very useful. A "Reset to defaults" link clears the saved version and reloads the original grocery list, in case anyone wants to start over.

A Few Notes on What I Took From This

- There's a real difference between a loop that displays data once and a UI that has to keep displaying it correctly as the data keeps changing — the second one needed a plan for what gets rebuilt and what doesn't.
- Re-rendering everything is the easy way out, but it breaks small things like focus and cursor position that people notice immediately, even if they can't say why it feels wrong.
- Giving each array item a stable id, separate from its position in the array, made add/remove reliable in a way that index-based matching wouldn't have been.
- A visual direction grounded in the actual subject — a ledger and a receipt, for a shopping list — gave the design a reason for its choices instead of just picking colors that looked nice.

References Used

Same references as the original task, revisited for the array and loop mechanics underneath the new interface.

Article — Arrays: [w3schools.com/js/js_arrays.asp](https://www.w3schools.com/js/js_arrays.asp)



Article — Loops: w3schools.com/js/js_loops.asp

Video 1: youtube.com/watch?v=IFZZAaiatcQ

Video 2: youtube.com/watch?v=y32sWmu-RI4

Files Submitted

- script.js — the editable shoppingList array, the for loop that renders it, the while loop that totals it, and the add/edit/remove/save logic.
- index.html — the redesigned "Grocery Ledger" interface: an editable ledger table and a live receipt-style summary panel.

THANK YOU!